

MAKALAH
“TUGAS 4 SISTEM OPRASI”

Disusun Untuk Memenuhi Tugas Mata Kuliah
Sistem Oprasi

Dosen : Triawan Adi Cahyanto, M.Kom



Disusun Oleh:
Nama: Fajar Satria nusaputra
NIM:2410651058
Prodi:Teknik Informatika
Kelas: C

UNIVERSITAS MUHAMMADIYAH JEMBER
2025-2026

BAB I EKSPERIMEN DENGAN DATA YANG BERBEDA

Jalankan setiap algoritma dengan data berikut dan catat hasilnya:

Data Set 1:

1. P1: AT=0, BT=10
2. P2: AT=1, BT=5
3. P3: AT=2, BT=8

Data Set 2:

1. P1: AT=0, BT=5
2. P2: AT=1, BT=3
3. P3: AT=2, BT=8
4. P4: AT=3, BT=6

1. FCFS

Proses dijalankan berdasarkan urutan kedatangan (Arrival Time), Proses yang pertama datang akan dijalankan sampai selesai sebelum proses lain dijalankan.

Data Set 1

```
koch@Kchng:~$ ./fcfs
=== SIMULASI ALGORITMA FCFS ===
Masukkan jumlah proses: 3

Proses P1
Arrival Time: 0
Burst Time: 10

Proses P2
Arrival Time: 1
Burst Time: 5

Proses P3
Arrival Time: 2
Burst Time: 8

=== HASIL PENJADWALAN FCFS ===
PID    AT    BT    CT    TAT    WT
---    --    --    --    ---    --
P1      0     10    10     10     0
P2      1      5    15     14     9
P3      2      8    23     21    13

Average Turnaround Time: 15.00
Average Waiting Time: 7.33

=== GANTT CHART ===
| P1 | P2 | P3 |
0 10 15 23
koch@Kchng:~$ |
```

Data Set 2

```
koch@Kchng:~$ ./fcfs
=== SIMULASI ALGORITMA FCFS ===
Masukkan jumlah proses: 4

Proses P1
Arrival Time: 0
Burst Time: 5

Proses P2
Arrival Time: 1
Burst Time: 3

Proses P3
Arrival Time: 2
Burst Time: 8

Proses P4
Arrival Time: 3
Burst Time: 6

=== HASIL PENJADWALAN FCFS ===
PID  AT   BT   CT   TAT  WT
---  --   --   --   ---  --
P1    0    5    5    5    0
P2    1    3    8    7    4
P3    2    8   16   14    6
P4    3    6   22   19   13

Average Turnaround Time: 11.25
Average Waiting Time: 5.75

=== GANTT CHART ===
| P1 | P2 | P3 | P4 |
0 5 8 16 22
koch@Kchng:~$ |
```

2. SJF

Proses yang punya waktu paling singkat dikerjakan lebih dulu (dari proses yang sudah datang).

Data Set 1

```
koch@Kchng:~$ ./sjf
=== SIMULASI ALGORITMA SJF (Non-Preemptive) ===
Masukkan jumlah proses: 3

Proses P1
Arrival Time: 0
Burst Time: 10

Proses P2
Arrival Time: 1
Burst Time: 5

Proses P3
Arrival Time: 2
Burst Time: 8

=== HASIL PENJADWALAN SJF ===
PID  AT   BT   CT   TAT  WT
---  --   --   --   ---  --
P1    0   10   10   10    0
P2    1    5   15   14    9
P3    2    8   23   21   13

Average Turnaround Time: 15.00
Average Waiting Time: 7.33

=== URUTAN EKSEKUSI ===
Proses dieksekusi dalam urutan: P1 -> P2 -> P3
koch@Kchng:~$ |
```

Data Set 2

```

koch@Kchng:~$ ./sjf
=== SIMULASI ALGORITMA SJF (Non-Preemptive) ===
Masukkan jumlah proses: 4

Proses P1
Arrival Time: 0
Burst Time: 5

Proses P2
Arrival Time: 1
Burst Time: 3

Proses P3
Arrival Time: 2
Burst Time: 8

Proses P4
Arrival Time: 3
Burst Time: 6

=== HASIL PENJADWALAN SJF ===
PID  AT  BT  CT  TAT  WT
---  --  --  --  ---  --
P1    0   5   5   5    0
P2    1   3   8   7    4
P3    2   8  22  20   12
P4    3   6  14  11    5

Average Turnaround Time: 10.75
Average Waiting Time: 5.25

=== URUTAN EKSEKUSI ===
Proses dieksekusi dalam urutan: P1 -> P2 -> P4 -> P3
koch@Kchng:~$

```

3. Round Robin

Semua proses bergantian mendapatkan waktu CPU selama 3 detik, kalau belum selesai, nanti akan lanjut di giliran berikutnya.

Data Set 1

```

koch@Kchng:~$ ./round_robin
=== SIMULASI ALGORITMA ROUND ROBIN ===
Masukkan jumlah proses: 3
Masukkan time quantum: 3

Proses P1
Arrival Time: 0
Burst Time: 10

Proses P2
Arrival Time: 1
Burst Time: 5

Proses P3
Arrival Time: 2
Burst Time: 8

=== PROSES EKSEKUSI ===
Waktu 0: Menjalankan P1 -> Sisa waktu: 7
Waktu 3: Menjalankan P1 -> Sisa waktu: 4
Waktu 6: Menjalankan P2 -> Sisa waktu: 2
Waktu 9: Menjalankan P3 -> Sisa waktu: 5
Waktu 12: Menjalankan P1 -> Sisa waktu: 1
Waktu 15: Menjalankan P1 -> Selesai pada waktu 16
Waktu 16: Menjalankan P1 -> Selesai pada waktu 16
Waktu 16: Menjalankan P2 -> Selesai pada waktu 18

=== HASIL PENJADWALAN ROUND ROBIN ===
PID  AT  BT  CT  TAT  WT
---  --  --  --  ---  --
P1    0  10  16  16    6
P2    1   5  18  17   12
P3    2   8 -1437725280 22499 -1437736503

Average Turnaround Time: 7510.67
Average Waiting Time: -479245472.00
koch@Kchng:~$

```

Data Set 2

```

koch@Kchnq:~$ ./round_robin
=== SIMULASI ALGORITMA ROUND ROBIN ===
Masukkan jumlah proses: 4
Masukkan time quantum: 4

Proses P1
Arrival Time: 0
Burst Time: 5

Proses P2
Arrival Time: 1
Burst Time: 3

Proses P3
Arrival Time: 2
Burst Time: 8

Proses P4
Arrival Time: 3
Burst Time: 6

=== PROSES EKSEKUSI ===
Waktu 0: Menjalankan P1 -> Sisa waktu: 1
Waktu 4: Menjalankan P1 -> Selesai pada waktu 5
Waktu 5: Menjalankan P2 -> Selesai pada waktu 8
Waktu 8: Menjalankan P3 -> Sisa waktu: 4
Waktu 12: Menjalankan P4 -> Sisa waktu: 2
Waktu 16: Menjalankan P1 -> Selesai pada waktu 16
Waktu 16: Menjalankan P3 -> Selesai pada waktu 20

=== HASIL PENJADWALAN ROUND ROBIN ===

```

PID	AT	BT	CT	TAT	WT
P1	0	5	16	16	11
P2	1	3	8	7	4
P3	2	8	20	18	10
P4	3	6	22113	-1229004183	22113

```

Average Turnaround Time: -307251040.00
Average Waiting Time: 5534.50
koch@Kchnq:~$

```

Kesimpulannya:

Jika mau efisien dan cepat selesai, pakai SJF.

Jika mau sederhana dan adil urut datang, pakai FCFS.

Jika mau adil untuk semua (interaktif), pakai Round Robin.

Jadi, FCFS itu seperti antrian kasir, kalau SJF melayani yang paling sedikit agar cepat selesai, dan jika round robin seperti gantian main computer.

BAB II

ANALISIS EKSPERIMEN KETIGA ALGORITMA

Jawab pertanyaan berikut:

1. Algoritma mana yang memberikan Average Waiting Time terkecil?
2. Mengapa hasil algoritma berbeda-beda?
3. Kapan sebaiknya menggunakan algoritma FCFS?
4. Kapan sebaiknya menggunakan algoritma SJF?
5. Kapan sebaiknya menggunakan algoritma Round Robin?

Jawaban:

1. Untuk data set 1 dengan Average Waiting Time terkecil adalah FCFS dan SJF dengan waktu yang sama yaitu 7.33 .
Untuk data set 2 dengan Average Waiting Time terkecil adalah SJF dengan waktu tunggu rata rata 5.25
2. Hasilnya berbeda karena setiap algoritma punya cara masing masing dalam mengatur urutan proses.FCFS menjalankan proses berdasarkan urutan datangnya,jika SJF fokus pada waktu / pekerjaan yang paling singkat terlebih dahulu, sedangkan Round Robin fokus pada pembagian waktu dengan adil untuk semua proses.
3. Algoritma FCFS berkerja seperti antrian kasir maksudnya adalah siapa yang datang duluan maka akan dilayani duluan ,jadi sebaiknya jika ingin menggunakan algoritma FCFS ketika sistem sederhana ,prosesnya datangnya bergantian,dan tidak butuh respon cepat maka akan cocok dengan algoritma FCFS.
4. Proses yang punya waktu paling singkat dikerjakan lebih dulu,jadi sebaiknya jika ingin menggunakan algoritma SJF ketika prosesnya batch ,taahu perkiraan waktu setiap proses ,dan jika anda ingin sistem berjalan lebih efisien dan cepat.
5. Sebaiknya jika ingin menggunakan round robin saat sistem harus responsif,terutama jika ingin sitem yang multitasking dan interaktif yang mana semua proses bergiliran.

BAB III MODIFIKASI PROGRAM

Tambahkan fitur berikut pada salah satu program:

1. Tampilkan Gantt Chart yang lebih detail
2. Simpan hasil ke file text
3. Baca input dari file

1. FCFS

Buat file nano fcfs.txt

```
3
P1 0 24
P2 1 3
P3 2 6
```

Perbaiki kode nano modif_fcfs.c

```
#include <stdio.h>
#include <stdlib.h>

struct Process {
    char id[5];
    int arrival, burst;
    int waiting, turnaround, completion;
};

int main() {
    FILE *fin, *fout;
    fin = fopen("fcfs.txt", "r");
    fout = fopen("hasil_fcfs.txt", "w");

    if (fin == NULL) {
        printf("Gagal membuka file input.txt!\n");
        return 1;
    }

    int n;
    fscanf(fin, "%d", &n);

    struct Process p[n];

    for (int i = 0; i < n; i++) {
        fscanf(fin, "%s %d %d", p[i].id, &p[i].arrival, &p[i].burst);
    }

    fclose(fin);
```

```

// FCFS Scheduling
int current_time = 0;
float total_wt = 0, total_tat = 0;

printf("\n=== Hasil Penjadwalan FCFS ===\n");
fprintf(fout, "=== Hasil Penjadwalan FCFS ===\n");

printf("\nTabel Hasil:\n");
printf("ID\tAT\tBT\tCT\tTAT\tWT\n");
fprintf(fout, "\nID\tAT\tBT\tCT\tTAT\tWT\n");

for (int i = 0; i < n; i++) {
    if (current_time < p[i].arrival)
        current_time = p[i].arrival;

    p[i].completion = current_time + p[i].burst;
    p[i].turnaround = p[i].completion - p[i].arrival;
    p[i].waiting = p[i].turnaround - p[i].burst;
    current_time = p[i].completion;

    total_wt += p[i].waiting;
    total_tat += p[i].turnaround;

    printf("%s\t%d\t%d\t%d\t%d\t%d\n",
           p[i].id, p[i].arrival, p[i].burst,
           p[i].completion, p[i].turnaround, p[i].waiting);
    fprintf(fout, "%s\t%d\t%d\t%d\t%d\t%d\n",
           p[i].id, p[i].arrival, p[i].burst,
           p[i].completion, p[i].turnaround, p[i].waiting);
}

float avg_wt = total_wt / n;
float avg_tat = total_tat / n;

printf("\nRata-rata Waiting Time: %.2f", avg_wt);
printf("\nRata-rata Turnaround Time: %.2f\n", avg_tat);
fprintf(fout, "\nRata-rata Waiting Time: %.2f", avg_wt);
fprintf(fout, "\nRata-rata Turnaround Time: %.2f\n", avg_tat);

// Gantt Chart lebih detail
printf("\nGantt Chart:\n|");
fprintf(fout, "\nGantt Chart:\n|");

int start = 0;
for (int i = 0; i < n; i++) {
    if (i == 0)
        start = p[i].arrival;
    printf(" %s (%d-%d) |", p[i].id, start, p[i].completion);
    fprintf(fout, " %s (%d-%d) |", p[i].id, start, p[i].completion);
    start = p[i].completion;
}

printf("\n\nHasil juga disimpan di file hasil_fcfs.txt\n");
fprintf(fout, "\n");

fclose(fout);
return 0;
}

```


Program ini akan membaca data dari file FCFS.txt.

Setiap proses akan disimpan dalam array struct Process yang berisi ID, arrival time, burst time, dll.

CPU mulai dari waktu 0, dan mengecek proses mana yang sudah datang.

Proses dijalankan satu per satu sesuai urutan.

Setelah selesai, program menghitung:

- CT (Completion Time) = waktu proses selesai
- TAT (Turnaround Time) = CT – AT
- WT (Waiting Time) = TAT – BT

Semua hasil akan ditampilkan dan di simpan ke file hasil_fcfs.txt

Outputnya:

```
koch@kchng:~$ nano fcfs.txt
koch@kchng:~$ nano modif_fcfs.c
koch@kchng:~$ gcc modif_fcfs.c modif_fcfs
/usr/bin/ld: cannot find modif_fcfs: No such file or directory
collect2: error: ld returned 1 exit status
koch@kchng:~$ gcc modif_fcfs.c -o modif_fcfs
koch@kchng:~$ ./modif_fcfs

=== Hasil Penjadwalan FCFS ===

Tabel Hasil:
  ID  AT   BT   CT   TAT  WT
P1   0   24   24   24   0
P2   1    3   27   26   23
P3   2    6   33   31   25

Rata-rata Waiting Time: 16.00
Rata-rata Turnaround Time: 27.00

Gantt Chart:
| P1 (0-24) | P2 (24-27) | P3 (27-33) |

Hasil juga disimpan di file hasil_fcfs.txt
koch@kchng:~$
```

2. SJF

Buat file nano SJF.txt

```
2
P1 2 4
P2 3 5
```

Perbaiki kode SJF.c

```
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>

struct Process {
    char id[5];
    int arrival, burst;
    int waiting, turnaround, completion;
```

```

    bool done;
};

int main() {
    FILE *fin = fopen("sjf.txt", "r");
    FILE *fout = fopen("hasil_sjf.txt", "w");

    if (!fin) {
        printf("Gagal membuka file input.txt!\n");
        return 1;
    }

    int n;
    fscanf(fin, "%d", &n);
    struct Process p[n];

    for (int i = 0; i < n; i++) {
        fscanf(fin, "%s %d %d", p[i].id, &p[i].arrival, &p[i].burst);
        p[i].done = false;
    }
    fclose(fin);

    int completed = 0, current_time = 0;
    float total_wt = 0, total_tat = 0;

    printf("\n=== Hasil Penjadwalan SJF ===\n");
    fprintf(fout, "=== Hasil Penjadwalan SJF ===\n");

    printf("\nTabel Hasil:\nID\tAT\tBT\tCT\tTAT\tWT\n");
    fprintf(fout, "\nID\tAT\tBT\tCT\tTAT\tWT\n");

    printf("\nGantt Chart:\n|");
    fprintf(fout, "\nGantt Chart:\n|");

    while (completed < n) {
        int idx = -1, min_bt = 9999;

        for (int i = 0; i < n; i++) {
            if (!p[i].done && p[i].arrival <= current_time) {
                if (p[i].burst < min_bt) {
                    min_bt = p[i].burst;
                    idx = i;
                }
            }
        }

        if (idx == -1) {
            current_time++;
            continue;
        }

        int start = current_time;
        current_time += p[idx].burst;
        p[idx].completion = current_time;
        p[idx].turnaround = p[idx].completion - p[idx].arrival;
        p[idx].waiting = p[idx].turnaround - p[idx].burst;
        p[idx].done = true;
        completed++;

        printf(" %s (%d-%d) |", p[idx].id, start, current_time);
    }
}

```

```

        fprintf(fout, " %s (%d-%d) |", p[idx].id, start, current_time);

        total_wt += p[idx].waiting;
        total_tat += p[idx].turnaround;
    }

    printf("\n\nID\tAT\tBT\tCT\tTAT\tWT\n");
    fprintf(fout, "\n\nID\tAT\tBT\tCT\tTAT\tWT\n");

    for (int i = 0; i < n; i++) {
        printf("%s\t%d\t%d\t%d\t%d\t%d\n", p[i].id, p[i].arrival, p[i].burst,
            p[i].completion, p[i].turnaround, p[i].waiting);
        fprintf(fout, "%s\t%d\t%d\t%d\t%d\t%d\n", p[i].id, p[i].arrival, p[i].burst,
            p[i].completion, p[i].turnaround, p[i].waiting);
    }

    printf("\nRata-rata Waiting Time: %.2f", total_wt / n);
    printf("\nRata-rata Turnaround Time: %.2f\n", total_tat / n);
    fprintf(fout, "\nRata-rata Waiting Time: %.2f", total_wt / n);
    fprintf(fout, "\nRata-rata Turnaround Time: %.2f\n", total_tat / n);

    fclose(fout);
    printf("\n\nHasil juga disimpan di file hasil_sjf.txt\n");
    return 0;
}

```

Penjelasan:

- ❖ Program membaca data dari SJF.txt.
- ❖ Semua proses akan disimpan dan diberi tanda apakah proses tersebut sudah selesai atau belum (bool done).
- ❖ CPU akan memilih proses yang sudah datang dan memiliki burst time terkecil.
- ❖ Setelah satu proses selesai, dihitung CT, TAT, WT, lalu lanjut ke proses berikutnya.
- ❖ Program menampilkan Gantt Chart yang menunjukkan waktu mulai dan selesai setiap proses.
- ❖ Hasil akhir akan disimpan di file hasil_sjf.txt

Outputnya:

```

kech@Kchng:~$ nano modif_sjf.c
kech@Kchng:~$ gcc modif_sjf.c -o modif_sjf
kech@Kchng:~$ ./modif_sjf

=== Hasil Penjadwalan SJF ===

Tabel Hasil:
ID      AT      BT      CT      TAT      WT
P1      2        4        6        4        0
P2      3        5       11        8        3

Gantt Chart:
| P1 (2-6) | P2 (6-11) |

ID      AT      BT      CT      TAT      WT
P1      2        4        6        4        0
P2      3        5       11        8        3

Rata-rata Waiting Time: 1.50
Rata-rata Turnaround Time: 6.00

Hasil juga disimpan di file hasil_sjf.txt
kech@Kchng:~$

```

3. Round Robin

Buat file Round_Robin.txt

```
2
P1 2 3
P2 1 5
```

Perbaiki kode Round_Robin.c

```
#include <stdio.h>
#include <stdlib.h>

struct Process {
    char id[5];
    int arrival, burst, remaining;
    int waiting, turnaround, completion;
};

int main() {
    FILE *fin = fopen("Round_Robin.txt", "r");
    FILE *fout = fopen("hasil_rr.txt", "w");

    if (!fin) {
        printf("Gagal membuka file input.txt!\n");
        return 1;
    }

    int n, quantum = 16;
    fscanf(fin, "%d", &n);

    struct Process p[n];
    for (int i = 0; i < n; i++) {
        fscanf(fin, "%s %d %d", p[i].id, &p[i].arrival, &p[i].burst);
        p[i].remaining = p[i].burst;
    }
    fclose(fin);

    int completed = 0, current_time = 0;
    float total_wt = 0, total_tat = 0;
    int done[n];
    for (int i = 0; i < n; i++) done[i] = 0;

    printf("\n=== Hasil Penjadwalan Round Robin (Q=16) ===\n");
    fprintf(fout, "=== Hasil Penjadwalan Round Robin (Q=16) ===\n");

    printf("\nGantt Chart:\n|");
    fprintf(fout, "\nGantt Chart:\n|");

    while (completed < n) {
        int idle = 1;
        for (int i = 0; i < n; i++) {
            if (p[i].arrival <= current_time && p[i].remaining > 0) {
                idle = 0;
                int start = current_time;
                if (p[i].remaining <= quantum) {
                    current_time += p[i].remaining;
                    p[i].remaining = 0;
                    p[i].completion = current_time;
                    completed++;
                }
            }
        }
        if (idle) {
            current_time++;
        }
    }

    printf("\nTotal Waktu Tunggu: %.2f\n", total_wt);
    fprintf(fout, "\nTotal Waktu Tunggu: %.2f\n", total_wt);

    printf("\nTotal Turnaround Time: %.2f\n", total_tat);
    fprintf(fout, "\nTotal Turnaround Time: %.2f\n", total_tat);
}
```

```

        } else {
            current_time += quantum;
            p[i].remaining -= quantum;
        }

        printf(" %s (%d-%d) |", p[i].id, start, current_time);
        fprintf(fout, " %s (%d-%d) |", p[i].id, start, current_time);
    }
}

if (idle) current_time++;
}

printf("\n\nID\tAT\tBT\tCT\tTAT\tWT\n");
fprintf(fout, "\n\nID\tAT\tBT\tCT\tTAT\tWT\n");

for (int i = 0; i < n; i++) {
    p[i].turnaround = p[i].completion - p[i].arrival;
    p[i].waiting = p[i].turnaround - p[i].burst;
    total_tat += p[i].turnaround;
    total_wt += p[i].waiting;

    printf("%s\t%d\t%d\t%d\t%d\t%d\n", p[i].id, p[i].arrival, p[i].burst,
        p[i].completion, p[i].turnaround, p[i].waiting);
    fprintf(fout, "%s\t%d\t%d\t%d\t%d\t%d\n", p[i].id, p[i].arrival, p[i].burst,
        p[i].completion, p[i].turnaround, p[i].waiting);
}

printf("\nRata-rata Waiting Time: %.2f", total_wt / n);
printf("\nRata-rata Turnaround Time: %.2f\n", total_tat / n);
fprintf(fout, "\nRata-rata Waiting Time: %.2f", total_wt / n);
fprintf(fout, "\nRata-rata Turnaround Time: %.2f\n", total_tat / n);

fclose(fout);
printf("\n\nHasil juga disimpan di file hasil_rr.txt\n");
return 0;
}

```

Penjelasan:

Program membaca jumlah proses dan datanya dari Round_Robin.txt.

Setiap proses mempunyai variabel remaining untuk menyimpan sisa waktu burst.

CPU akan mengeksekusi setiap proses selama quantum (3 satuan waktu).

Jika proses selesai, dicatat CT, TAT, dan WT.

Proses yang belum selesai akan kembali masuk antrian.

Program menampilkan Gantt Chart yang memperlihatkan urutan bergantian tiap proses.

Hasil akhir akan disimpan ke hasil_rr.txt

Outputnya:

```
kech@Kching:~$ nano Round_Robin.txt
kech@Kching:~$ nano modif_Round_Robin.c
kech@Kching:~$ gcc modif_Round_Robin.c -o modif_Round_Robin
kech@Kching:~$ ./modif_Round_Robin
```

=== Hasil Penjadwalan Round Robin (Q=16) ===

Gantt Chart:

| P2 (1-6) | P1 (6-9) |

ID	AT	BT	CT	TAT	WT
P1	2	3	9	7	4
P2	1	5	6	5	0

Rata-rata Waiting Time: 2.00

Rata-rata Turnaround Time: 6.00

Hasil juga disimpan di file hasil_rr.txt

```
kech@Kching:~$ |
```

BAB IV

EKSPERIMEN DENGAN DATA BERBEDA UNTUK PERBANDINGAN ALGORITMA

Test Case 1: Proses dengan burst time bervariasi

Jumlah proses: 4

Time quantum: 3

P1: AT=0, BT=24

P2: AT=1, BT=3

P3: AT=2, BT=6

P4: AT=3, BT=6

FCFS

Ganti kode FCFS.txt

```
4
P1 0 24
P2 1 3
P3 2 6
P4 3 6
```

Outputnya akan menjadi seperti ini:

```
koch@kchng:~/hano1$ cat test.txt
koch@kchng:~$ ./modif_fcfs

=== Hasil Penjadwalan FCFS ===

Tabel Hasil:
ID  AT   BT   CT   TAT  WT
P1   0   24   24   24   0
P2   1    3   27   26   23
P3   2    6   33   31   25
P4   3    6   39   36   30

Rata-rata Waiting Time: 19.50
Rata-rata Turnaround Time: 29.25

Gantt Chart:
| P1 (0-24) | P2 (24-27) | P3 (27-33) | P4 (33-39) |

Hasil juga disimpan di file hasil_fcfs.txt
koch@kchng:~$
```

SJF

```
4
P1 0 24
P2 1 3
P3 2 6
P4 3 6
```

Outputnya akan menjadi seperti ini

```

koch@Kchng:~$ ./modif_sjf

=== Hasil Penjadwalan SJF ===

Tabel Hasil:
ID      AT      BT      CT      TAT      WT
Gantt Chart:
| P1 (0-24) | P2 (24-27) | P3 (27-33) | P4 (33-39) |

ID      AT      BT      CT      TAT      WT
P1       0      24      24      24       0
P2       1       3      27      26      23
P3       2       6      33      31      25
P4       3       6      39      36      30

Rata-rata Waiting Time: 19.50
Rata-rata Turnaround Time: 29.25

Hasil juga disimpan di file hasil_sjf.txt
koch@Kchng:~$

```

Round Robin

Ganti kode Round_Robin.txt

```

4
P1 0 24
P2 1 3
P3 2 6
P4 3 6

```

Ganti quantum di file modifikasi_Round_Robin

```

int n, quantum = 3;
fscanf(fin, "%d", &n);

struct Process p[n];
for (int i = 0; i < n; i++) {
    fscanf(fin, "%s %d %d", p[i].id, &p[i].arrival, &p[i].burst);
    p[i].remaining = p[i].burst;
}
fclose(fin);

int completed = 0, current_time = 0;
float total_wt = 0, total_tat = 0;
int done[n];
for (int i = 0; i < n; i++) done[i] = 0;

printf("\n=== Hasil Penjadwalan Round Robin (Q=3) ===\n");
fprintf(fout, "=== Hasil Penjadwalan Round Robin (Q=3) ===\n");

```

Outputnya akan seperti:


```

koch@KChng:~$ gcc modif_Round_Robin.c -o modif_Round_Robin
koch@KChng:~$ ./modif_Round_Robin

=== Hasil Penjadwalan Round Robin (Q=3) ===

Gantt Chart:
| P1 (0-3) | P2 (3-6) | P3 (6-9) | P4 (9-12) | P1 (12-15) | P3 (15-18) | P4 (18-21) | P1 (21-24) | P1 (24-27) | P1 (27-30) | P1 (30-33) | P1 (33-36) | P1 (36-39) |

ID   AT   BT   CT   TAT  WT
P1   0    24   39   39   15
P2   1     3    6    5    2
P3   2     6   18   16   10
P4   3     6   21   18   12

Rata-rata Waiting Time: 9.75
Rata-rata Turnaround Time: 19.50

Hasil juga disimpan di file hasil_rr.txt
koch@KChng:~$

```

Test Case 2: Proses datang bersamaan

Jumlah proses: 5

Time quantum: 4

P1: AT=0, BT=10

P2: AT=0, BT=5

P3: AT=0, BT=8

P4: AT=0, BT=12

P5: AT=0, BT=6

FCFS

```

5
P1 0 10
P2 0 5
P3 0 8
P4 0 12
P5 0 6

```

Outputnya akan menjadi seperti ini:

```

koch@Kchng:~$ ./modif_fcfs

=== Hasil Penjadwalan FCFS ===

Tabel Hasil:
ID   AT   BT   CT   TAT   WT
P1   0    10   10   10    0
P2   0     5   15   15   10
P3   0     8   23   23   15
P4   0    12   35   35   23
P5   0     6   41   41   35

Rata-rata Waiting Time: 16.60
Rata-rata Turnaround Time: 24.80

Gantt Chart:
| P1 (0-10) | P2 (10-15) | P3 (15-23) | P4 (23-35) | P5 (35-41) |

Hasil juga disimpan di file hasil_fcfs.txt
koch@Kchng:~$

```

SJF

```

5
P1 0 10
P2 0 5
P3 0 8
P4 0 12
P5 0 6

```

```

koch@Kchng:~$ ./modif_sjf

=== Hasil Penjadwalan SJF ===

Tabel Hasil:
ID   AT   BT   CT   TAT   WT
P1   0    10   29   29   19
P2   0     5    5    5    0
P3   0     8   19   19   11
P4   0    12   41   41   29
P5   0     6   11   11    5

Rata-rata Waiting Time: 12.80
Rata-rata Turnaround Time: 21.00

Gantt Chart:
| P2 (0-5) | P5 (5-11) | P3 (11-19) | P1 (19-29) | P4 (29-41) |

Hasil juga disimpan di file hasil_sjf.txt
koch@Kchng:~$

```

Round Robin

```

5
P1 0 10
P2 0 5
P3 0 8
P4 0 12
P5 0 6

```

Outputnya akan menjadi seperti ini:

```

koch@Kchng:~$ ./modif_Round_Robin

=== Hasil Penjadwalan Round Robin (Q=4) ===

Gantt Chart:
| P1 (0-4) | P2 (4-8) | P3 (8-12) | P4 (12-16) | P5 (16-20) | P1 (20-24) | P2 (24-25) | P3 (25-29) | P4 (29-33) | P5 (33-35) | P1 (35-37) | P4 (37-41) |

ID   AT   BT   CT   TAT   WT
P1   0    10   37   37   27
P2   0     5   25   25   20
P3   0     8   29   29   21
P4   0    12   41   41   29
P5   0     6   35   35   29

Rata-rata Waiting Time: 25.20
Rata-rata Turnaround Time: 33.40

Hasil juga disimpan di file hasil_rr.txt
koch@Kchng:~$

```

Test Case 3: Proses datang dengan interval

Jumlah proses: 5

Time quantum: 2

- P1: AT=0, BT=8
- P2: AT=2, BT=4
- P3: AT=4, BT=9
- P4: AT=6, BT=5
- P5: AT=8, BT=3

FCFS

```
5
P1 0 8
P2 2 4
P3 4 9
P4 6 5
P5 8 3
```

Outputnya akan menjadi seperti ini

```
kech@Kchng:~$ ./modif_fcfs
=== Hasil Penjadwalan FCFS ===
Tabel Hasil:
ID  AT   BT   CT   TAT  WT
P1   0    8    8    8    0
P2   2    4   12   10    6
P3   4    9   21   17    8
P4   6    5   26   20   15
P5   8    3   29   21   18

Rata-rata Waiting Time: 9.40
Rata-rata Turnaround Time: 15.20

Gantt Chart:
| P1 (0-8) | P2 (8-12) | P3 (12-21) | P4 (21-26) | P5 (26-29) |
Hasil juga disimpan di file hasil_fcfs.txt
kech@Kchng:~$
```

SJF

```
5
P1 0 8
P2 2 4
P3 4 9
P4 6 5
P5 8 3
```

Outputnya akan menjadi seperti ini

```
koch@Kchng:~$ ./modif_sjf
=== Hasil Penjadwalan SJF ===

Tabel Hasil:
ID      AT      BT      CT      TAT      WT
Gantt Chart:
| P1 (0-8) | P5 (8-11) | P2 (11-15) | P4 (15-20) | P3 (20-29) |

ID      AT      BT      CT      TAT      WT
P1       0       8       8       8       0
P2       2       4       15      13       9
P3       4       9       29      25      16
P4       6       5       20      14       9
P5       8       3       11       3       0

Rata-rata Waiting Time: 6.80
Rata-rata Turnaround Time: 12.60

Hasil juga disimpan di file hasil_sjf.txt
koch@Kchng:~$ |
```

Round Robin

Ganti kode di file txt:

```
5
P1 0 8
P2 2 4
P3 4 9
P4 6 5
P5 8 3
```

Outputnya akan menjadi seperti ini

```
koch@Kchng:~$ ./modif_Round_Robin
=== Hasil Penjadwalan Round Robin (Q=2) ===

Gantt Chart:
| P1 (0-2) | P2 (2-4) | P3 (4-6) | P4 (6-8) | P5 (8-10) | P1 (10-12) | P2 (12-14) | P3 (14-16) | P4 (16-18) | P5 (18-19) | P1 (19-21) | P3 (21-23) | P4 (23-24) | P1 (24-26) | P3 (26-28) | P3 (28-29) |

ID      AT      BT      CT      TAT      WT
P1       0       8      26      26      18
P2       2       4      14      12       8
P3       4       9      29      25      16
P4       6       5      24      18      13
P5       8       3      19      11       8

Rata-rata Waiting Time: 12.60
Rata-rata Turnaround Time: 18.40

Hasil juga disimpan di file hasil_rr.txt
koch@Kchng:~$ |
```

Analisis dan Hasil:

FCFS

Proses ini akan dikerjakan berdasarkan urutan kedatangannya, lebih dari program ini adalah sederhana dan adil karena prosesnya sesuai dengan urutan kedatangannya, sedangkan kekurangannya prosesnya Panjang di awal dan juga dapat memperlambat proses lain.

Hasil pada test case 1 dan 3 ,proses yang datang pertama mendominasi waktu CPU sehingga proses lain akan menunggu lama.

SJF

Proses ini dikerjakan yang memiliki waktu eksekusi yang paling sedikit ,kelebihannya akan menghasilkan waktu rata rata yang paling kecil,kekurangannya jika proses yang Panjang terus tertunda,bisa terjadi starvation.

Hasilnya pada semua case ,algoritma ini merupakan algoritma yang paling efisien pada rata rata waktu tunggu.

Round Robin

Proses ini akan mendapatkan giliran CPU selama 'quantum' waktu yang ditentukan,kelebihan dari algoritma ini Adalah adil untuk semua proses dan responsif,sedangkan kekurangannya adalah jika quantum terlalu kecil proses sering berpindah dan menambah overhead.

Hasil

- Pada case 1 (quantum 3) proses yang pendek akan selesai lebih cepat ,akan tetapi proses yang Panjang perlu banyak giliran.
- Pada case 2 (quantum 4) pembagian waktu lebih seimbang dikarenakan semua proses datang secara bersamaan.
- Pada case 3 (quantum 2) system sangat responsif ,akan tetapi lebih banyak pergantian proses.

BAB V

ANALISIS EKSPERIMEN PERBANDINGAN ALGORITMA

1. Analisis Performa

- Dalam kondisi apa FCFS memberikan hasil terbaik?

FCFS akan memberikan hasil terbaik ketika semua proses memiliki waktu eksekusi yang sama dan datangnya berurutan. contohnya seperti pencetakan di printer.

- Mengapa SJF hampir selalu memberikan Average WT terkecil?

Karena SJF selalu memprioritaskan proses yang paling singkat terlebih dahulu, sehingga dapat mengurangi total waktu tunggu semua proses secara keseluruhan.

- Apa trade-off dari Round Robin?

Trade_off itu semakin kecil quantum maka sistem akan semakin responsif tapi akan banyak waktu yang terbuang untuk perghantian proses. jika semakin besar quantum sistem akan semakin efisien akan tetapi respon proses jadi lambat dan akan mirip seperti proses FCFS.

2. Context Switching

- Algoritma mana yang paling banyak melakukan context switch?

Round robin lah yang paling banyak melakukan context switch, karena setiap proses hanya berjalan selama quantum tertentu dan harus bergantian, sementara algoritma lain contohnya FCFS dan SJF itu akan mengeksekusi proses sampai selesai dan baru berpindah.

- Bagaimana pengaruh quantum terhadap jumlah context switch di Round Robin?

Pengaruh quantum terhadap jumlah context switch adalah semakin kecil quantumnya maka akan semakin banyak context switch yang terjadi, dan jika semakin besar quantumnya maka akan semakin sedikit context switchnya, akan tetapi sistem akan menjadi kurang responsif.

- Coba jalankan Round Robin dengan *quantum* 2, 4, 8, 16 - apa yang terjadi?

```
5
P1 0 8
P2 2 4
P3 4 9
P4 6 5
P5 8 3
```

Quantum 2

```
int n, quantum = 2;
fscanf(fin, "%d", &n);

struct Process p[n];
for (int i = 0; i < n; i++) {
    fscanf(fin, "%s %d %d", p[i].id, &p[i].arrival, &p[i].burst);
    p[i].remaining = p[i].burst;
}
fclose(fin);

int completed = 0, current_time = 0;
float total_wt = 0, total_tat = 0;
int done[n];
for (int i = 0; i < n; i++) done[i] = 0;

printf("\n=== Hasil Penjadwalan Round Robin (Q=2) ===\n");
fprintf(fout, "=== Hasil Penjadwalan Round Robin (Q=2) ===\n");
```

Outputnya:

```
koch@Kchng:~$ ./modif_Round_Robin

=== Hasil Penjadwalan Round Robin (Q=2) ===

Gantt Chart:
| P1 (0-2) | P2 (2-4) | P3 (4-6) | P4 (6-8) | P5 (8-10) | P1 (10-12) | P2 (12-14) | P3 (14-16) | P4 (16-18) | P5 (18-19) | P1 (19-21) | P3 (21-23) | P4 (23-24) | P1 (24-26) | P3 (26-28) | P3 (28-29) |

ID  AT  BT  CT  TAT  WT
P1   0   8   26  26   18
P2   2   4   14  12   8
P3   4   9   29  25   16
P4   6   5   24  18   13
P5   8   3   19  11   8

Rata-rata Waiting Time: 12.60
Rata-rata Turnaround Time: 18.40

Hasil juga disimpan di file hasil_rr.txt
koch@Kchng:~$ |
```

Quantum 4

```
int n, quantum = 4;
fscanf(fin, "%d", &n);

struct Process p[n];
for (int i = 0; i < n; i++) {
    fscanf(fin, "%s %d %d", p[i].id, &p[i].arrival, &p[i].burst);
    p[i].remaining = p[i].burst;
}
fclose(fin);

int completed = 0, current_time = 0;
float total_wt = 0, total_tat = 0;
int done[n];
for (int i = 0; i < n; i++) done[i] = 0;

printf("\n=== Hasil Penjadwalan Round Robin (Q=4) ===\n");
fprintf(fout, "=== Hasil Penjadwalan Round Robin (Q=4) ===\n");
```

Outputnya:

```

koch@Kchng:~$ ./modif_Round_Robin

=== Hasil Penjadwalan Round Robin (Q=4) ===

Gantt Chart:
| P1 (0-4) | P2 (4-8) | P3 (8-12) | P4 (12-16) | P5 (16-20) | P1 (20-24) | P2 (24-28) | P3 (28-32) | P4 (32-36) | P5 (36-40) |

ID  AT   BT   CT   TAT  WT
P1   0    4   10   37   27
P2   0    4    5   25   20
P3   0    8    8   29   21
P4   0   12   12   41   29
P5   0    6    6   35   29

Rata-rata Waiting Time: 25.20
Rata-rata Turnaround Time: 33.40

Hasil juga disimpan di file hasil_rr.txt
koch@Kchng:~$

```

Quantum 8

```

int n, quantum = 8;
fscanf(fin, "%d", &n);

struct Process p[n];
for (int i = 0; i < n; i++) {
    fscanf(fin, "%s %d %d", p[i].id, &p[i].arrival, &p[i].burst);
    p[i].remaining = p[i].burst;
}
fclose(fin);

int completed = 0, current_time = 0;
float total_wt = 0, total_tat = 0;
int done[n];
for (int i = 0; i < n; i++) done[i] = 0;

printf("\n=== Hasil Penjadwalan Round Robin (Q=8) ===\n");
fprintf(fout, "=== Hasil Penjadwalan Round Robin (Q=8) ===\n");

```

Outputnya:

```

koch@Kchng:~$ ./modif_Round_Robin

=== Hasil Penjadwalan Round Robin (Q=8) ===

Gantt Chart:
| P1 (0-8) | P2 (8-13) | P3 (13-21) | P4 (21-29) | P5 (29-35) | P1 (35-37) | P4 (37-41) |

ID  AT   BT   CT   TAT  WT
P1   0    8   10   37   27
P2   0    8    5   13    8
P3   0   13    8   21   13
P4   0   21   12   41   29
P5   0   29    6   35   29

Rata-rata Waiting Time: 21.20
Rata-rata Turnaround Time: 29.40

Hasil juga disimpan di file hasil_rr.txt
koch@Kchng:~$

```

Quantum 16

```

int n, quantum = 16;
fscanf(fin, "%d", &n);

struct Process p[n];
for (int i = 0; i < n; i++) {
    fscanf(fin, "%s %d %d", p[i].id, &p[i].arrival, &p[i].burst);
    p[i].remaining = p[i].burst;
}
fclose(fin);

int completed = 0, current_time = 0;
float total_wt = 0, total_tat = 0;
int done[n];
for (int i = 0; i < n; i++) done[i] = 0;

printf("\n=== Hasil Penjadwalan Round Robin (Q=16) ===\n");
fprintf(fout, "=== Hasil Penjadwalan Round Robin (Q=16) ===\n");

```

Outputnya:


```

koch@kchng:~$ ./modul_Round_Robin
=== Hasil Penjadwalan Round Robin (Q=16) ===
Gantt Chart:
| P1 (0-10) | P2 (10-15) | P3 (15-23) | P4 (23-35) | P5 (35-41) |
ID  AT   BT   CT   TAT  WT
P1   0   10   10   10   0
P2   0    5   15   15   10
P3   0    8   23   23   15
P4   0   12   35   35   23
P5   0    6   41   41   35

Rata-rata Waiting Time: 16.60
Rata-rata Turnaround Time: 24.80

Hasil juga disimpan di file hasil_rr.txt
koch@kchng:~$

```

Kesimpulannya

Berdasarkan proses di atas semakin kecil quantum maka sistem menjadi lebih responsif namun akan menjadi banyak context switch, sedangkan jika quantum lebih besar maka sistem akan menjadi lebih efisien namun akan menjadi kurang interaktif.

3. Fairness

- Algoritma mana yang paling "adil" untuk semua proses?

Algoritma yang paling adil adalah round robin karena setiap proses diberi kesempatan yang sama untuk berjalan dalam jatah waktu yang ditentukan (time quantum), sehingga tidak ada proses yang di tinggalkan ataupun didahulukan terus menerus.

- Proses mana yang paling dirugikan di algoritma FCFS?

Dalam FCFS proses yang paling dirugikan adalah proses yang datang belakangan dengan waktu eksekusi yang paling pendek, karena proses ini harus menunggu proses yang Panjang yang sudah datang duluan.

- Apakah ada kemungkinan starvation di SJF?

Pada algoritma SJF bisa menyebabkan starvation, terutama untuk proses besar yang datang lebih awal, karena proses yang kecil harus didahulukan. Untuk mencegah ini terjadi sistem bisa menggunakan Teknik aging agar proses lama tetap mendapatkan giliran.

4. Real World Application

- Untuk *web server* yang melayani banyak kecil, algoritma mana yang cocok?

Untuk web server yang melayani banyak permintaan kecil, algoritma yang cocok mungkin adalah round robin karena rr memberikan respon cepat dan adil untuk setiap request, sehingga semua pengguna mendapat pelayanan hampir bersamaan.

- Untuk *render video* (task besar), algoritma mana yang cocok?

Untuk render video (task besar), algoritma yang cocok mungkin adalah FCFS karena menjalankan proses besar secara penuh tanpa gangguan dan meminimalkan overhead dari pergantian proses.

- Untuk OS *desktop* (interaktif), algoritma mana yang cocok?

Untuk OS desktop, algoritma yang cocok mungkin adalah round robin karena memberikan respon cepat, keadilan dan memberikan kenyamanan bagi pengguna.